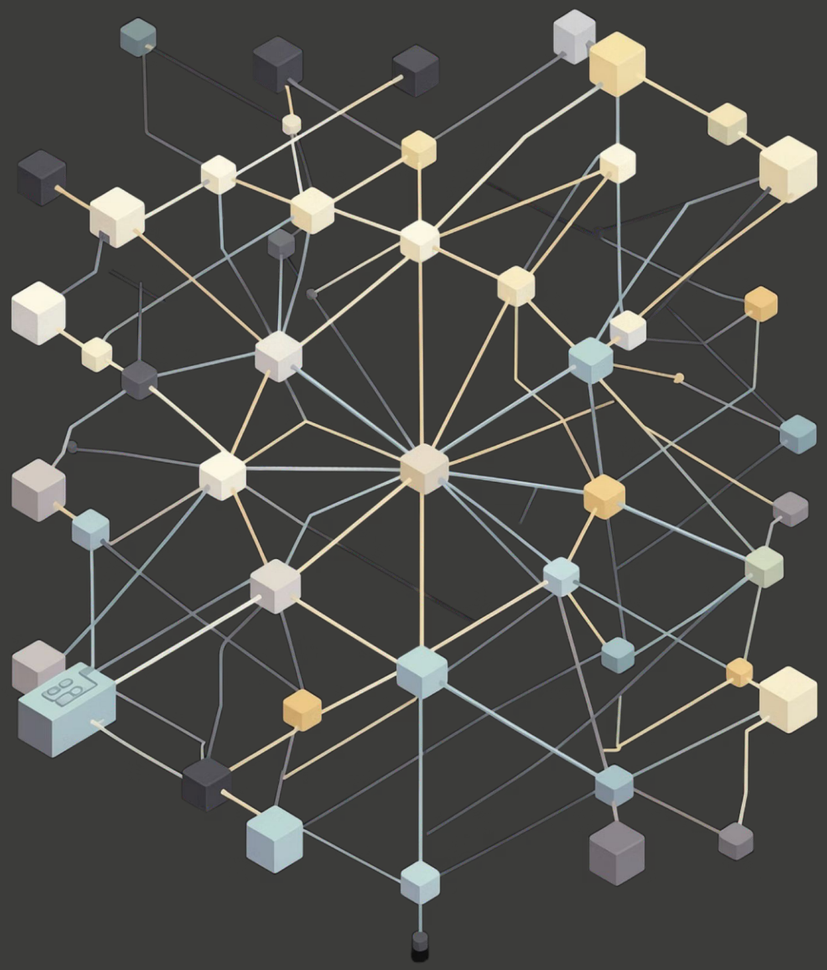


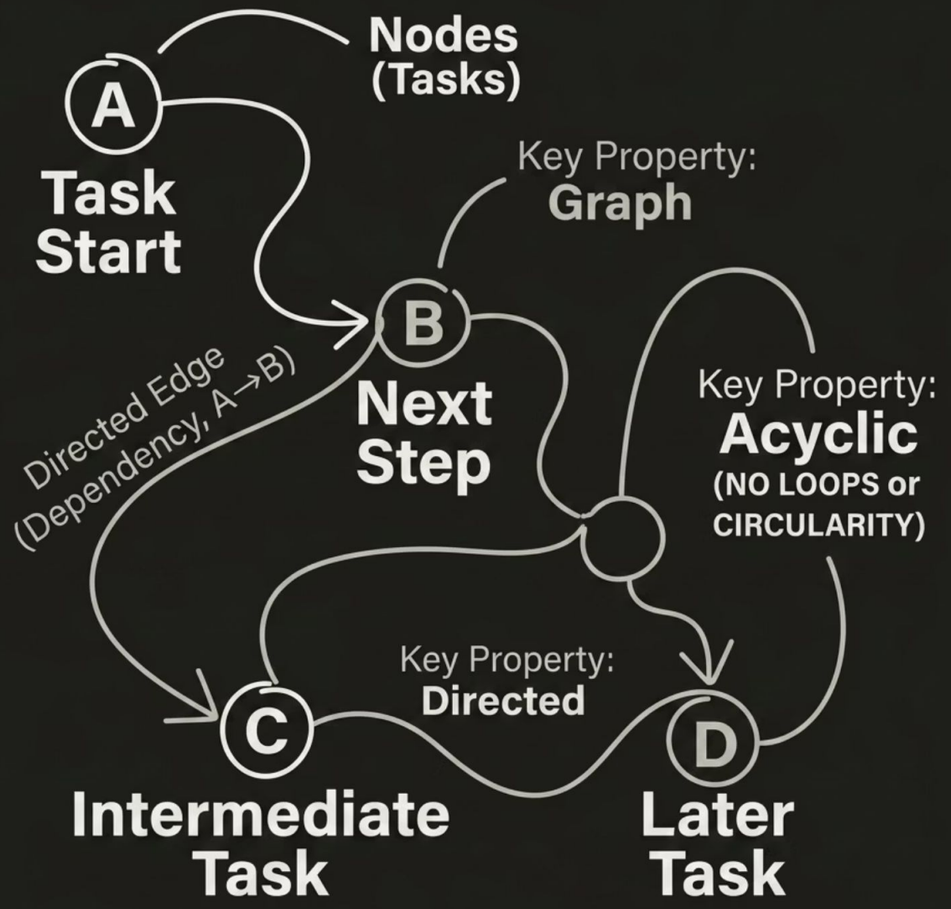
Team Name :ThreadX | Project :Parallel DAG Task Scheduler

> Parallel DAG-Based Task Scheduling with Dynamic Load Balancing using Work-Stealing in Shared Memory Systems

SYSTEMS PROGRAMMING | CONCURRENCY | PERFORMANCE

Mannya(102316067), Aditya(102496001), Gautam(102483024)

What is a DAG?



Directed Acyclic Graph

A DAG represents tasks and their dependencies. Each node is a task, and each edge shows the order of execution.

Directed: edges show task order

Acyclic: no cycle exists

Graph: can represent complex task dependencies

- Independent tasks can run in parallel

DAGs are ideal for parallel scheduling because independent nodes can execute simultaneously.

Why Parallel Scheduling?

Sequential execution wastes CPU resources. Parallel scheduling uses multiple threads to improve performance.

Throughput

More tasks run at the same time → Better CPU usage

Latency

Independent tasks run in parallel → Faster completion

Scalability

More cores/threads = Better performance

Work-Stealing: Smart Load Balancing

Dynamic Load Balancing

- Each thread has its own task queue (deque).

When a thread finishes its work and becomes **idle**, it **steals** a task from another busy thread's queue.

This keeps **all CPU cores busy** all the time.

Local Execution

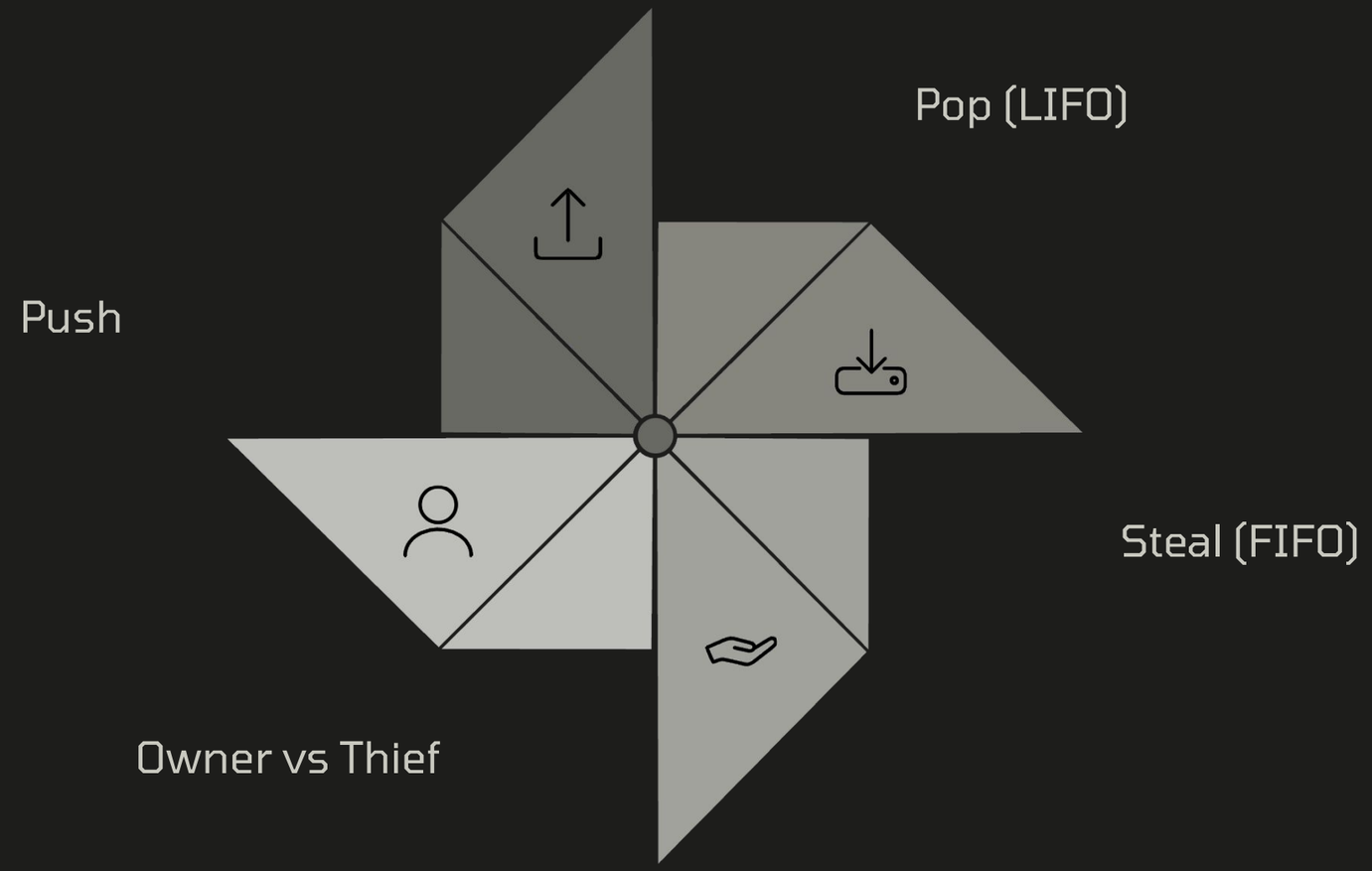
Threads pop tasks from their own deque (LIFO — fast, cache-friendly).

Stealing

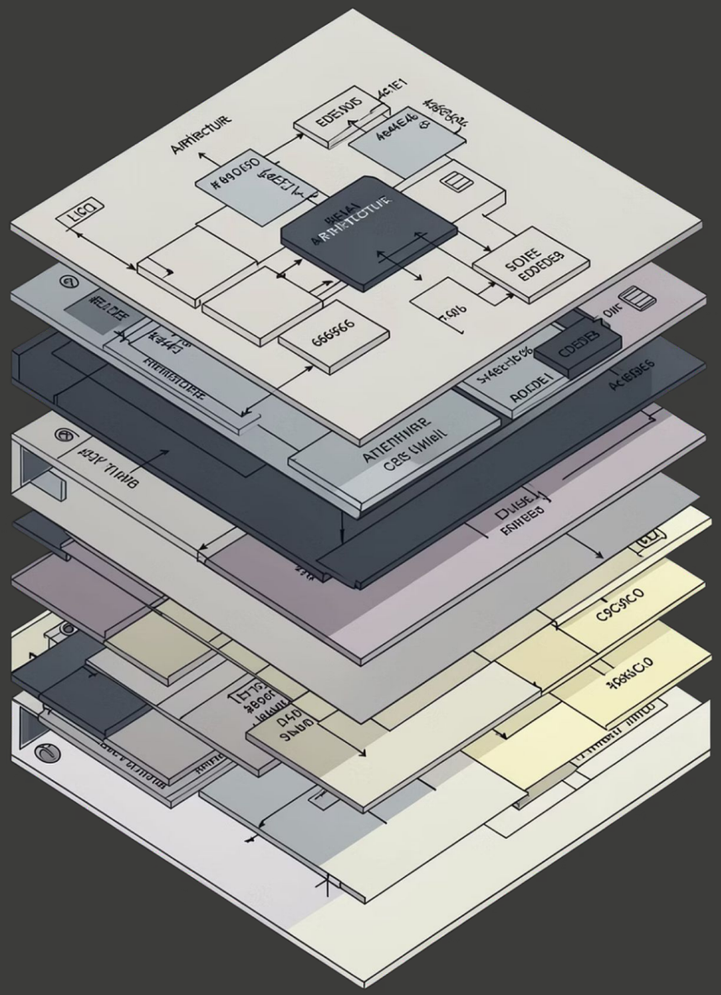
Takes tasks from bottom (reduces conflicts)

Why Double-Ended Queue (Deque)?

- One deque per thread
- Owner thread works from one end (LIFO — very fast)
- Other threads steal from the opposite end (FIFO).



This design reduces contention because the owner and the thief access different ends of the deque.



System Components

DAG Builder

Creates the task graph and stores task dependencies.

Thread Pool

Manages worker threads and their local task queues.

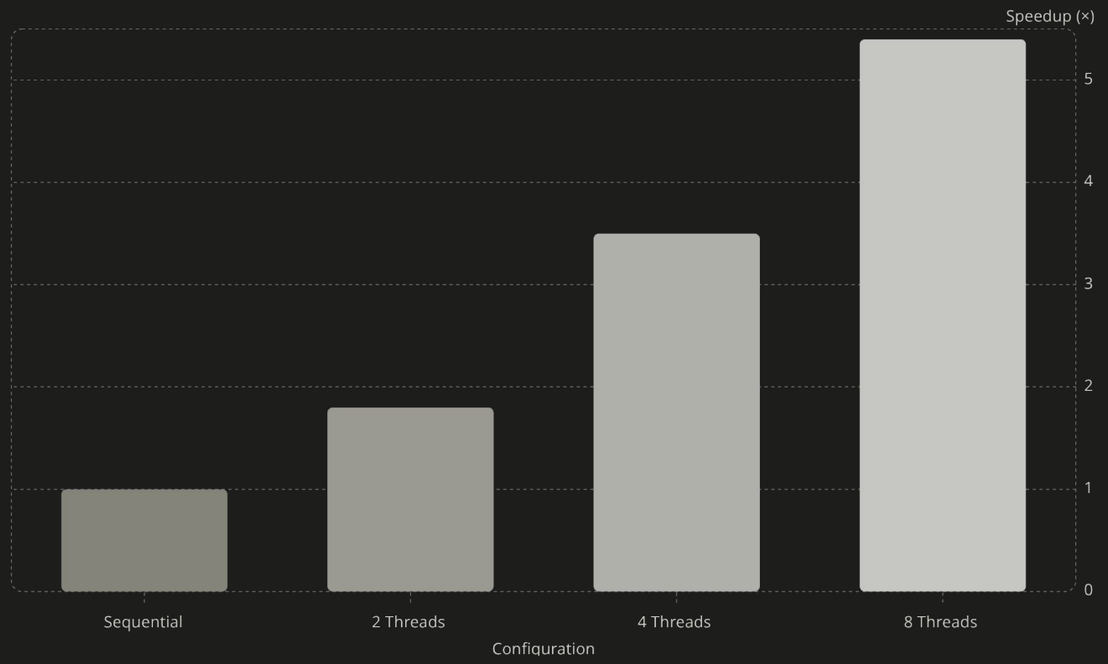
Scheduler Core

Assigns ready tasks and handles work-stealing during execution.

Performance Monitor

Measures execution time, speedup, and thread utilization.

Performance Results



Up to 7.04× Speedup

This project was compared with sequential execution across different thread counts. The results show improved speedup as the number of threads increases.

Speedup = Sequential Execution Time / Parallel Execution Time

Quick Summary

01

Model tasks as a DAG

Represent dependencies explicitly — no cycles, clean parallelism.

02

Assign per-thread deques

Each worker owns a double-ended queue for local task storage.

03

Execute locally, steal when idle

LIFO for owners, FIFO for thieves — low contention, high throughput.

04

Resolve dependencies dynamically

Tasks become ready only when all parent nodes complete.

05

Achieve 7.04× speedup

Near-linear scaling validates the architecture on real workloads.

Challenges & Future Scope

Current Challenges

- False sharing (cache line contention)
- Fine-grained tasks (too much overhead)
- Dependency storms (too many connections = less parallelism)

Future Directions

- Priority-based work-stealing
- NUMA-aware scheduling
- Dynamic graph changes
- Integration with OpenMP / Intel TBB